

Fine-Tuning a Large Language Model Using LoRA (PEFT)

Sentiment Analysis Project

Problem Statement

Large Language Models (LLMs) are powerful but computationally expensive to fine-tune completely. In real-world scenarios, organizations prefer Parameter-Efficient Fine-Tuning (PEFT) techniques such as LoRA (Low-Rank Adaptation) to adapt LLMs with minimal resources.

In this project, you will fine-tune a pretrained language model to perform sentiment classification (Positive / Negative) on movie reviews using the IMDb 50K Movie Reviews dataset from Kaggle. You will implement LoRA using the PEFT library and understand how modern LLMs are aligned using RLHF (conceptual understanding).

Dataset

- 50,000 movie reviews
- Labels: positive, negative
- Columns:
 - o **review** — movie review text
 - o **sentiment** — sentiment label

Dataset link:

https://docs.google.com/spreadsheets/d/1NhWmQq5Ec5H6LuoJE_pXtUJX5DNiFinexPYWD0Xa1ZM/

Step 1: Environment Setup

Install the required libraries and import all necessary packages.

1.1 Install Required Libraries

- transformers
- torch
- peft
- datasets

```
!pip install torch transformers peft datasets
```

1.2 Import Necessary Packages

```
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments, DataCollatorWithPadding
from peft import LoraConfig, get_peft_model
from datasets import Dataset
import pandas as pd
```

Step 2: Load the Dataset

Download the dataset, load the CSV file using pandas, and explore its contents.

2.1 Load the CSV File

- Download the dataset from the provided link
- Load the CSV file using pandas

```
#Loading Dataset
dataset = pd.read_csv("IMDB_Dataset.csv")
```

2.2 Explore the Dataset

- Display the first few rows

```
dataset.head()
```

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

- Display the dataset shape

```
dataset.shape
```

```
(50000, 2)
```

- Show distribution of sentiment labels

```
dataset['sentiment'].value_counts()
```

```
sentiment
positive    25000
negative    25000
Name: count, dtype: int64
```

Step 3: Data Preprocessing

Clean and prepare the data for model training.

3.1 Convert Sentiment Labels

- positive → 1
- negative → 0

```
imdb_data = dataset.copy()
```

```
#Converting sentiment labels : positive to 1 and negative to 0  
imdb_data['sentiment'] = pd.get_dummies(imdb_data['sentiment'], dtype=int, drop_first=True)
```

3.2 Train / Test Split

- Split the dataset into a Training set and a Testing set

```
#Converting the dataframe to HuggingFace Dataset  
imdb_dataset = Dataset.from_pandas(imdb_data)
```

```
#Splitting dataset into training and testing  
imdb_split_dataset = imdb_dataset.train_test_split(test_size=0.1, seed=42)
```

```
imdb_split_dataset
```

```
DatasetDict(  
  train: Dataset(  
    features: ['review', 'sentiment'],  
    num_rows: 45000  
  )  
  test: Dataset(  
    features: ['review', 'sentiment'],  
    num_rows: 5000  
  )  
)
```

Step 4: Load a Pretrained Model

Load a pretrained transformer model and perform a baseline prediction before fine-tuning.

4.1 Load Model and Tokenizer

- Load a pretrained transformer model (e.g., google/bert_uncased_L-2_H-128_A-2)
- Load the corresponding tokenizer

```
#Loading a pretrained model and its corresponding tokenizer
model_id = "google/bert_uncased_L-2_H-128_A-2"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForSequenceClassification.from_pretrained(model_id, num_labels=2)
```

Output:

```
config.json: 100% ██████████ 382/382 [00:00<00:00, 43.5kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
vocab.txt: 232k/? [00:00<00:00, 9.67MB/s]
model.safetensors: 100% ██████████ 17.7M/17.7M [00:01<00:00, 57.2MB/s]
Loading weights: 100% ██████████ 39/39 [00:00<00:00, 658.96it/s, Materializing param=bert.pooler.dense.weight]
```

4.2 Baseline Prediction (Before Fine-Tuning)

- Perform baseline predictions on a few sample reviews
- Record baseline results for later comparison

```
#Selecting first five reviews from test dataset
first_five_test_data = imdb_split_dataset['test'][0:5]
```

```
def predict_sentiment(texts, model, tokenizer):
    inputs = tokenizer(texts, return_tensors="pt", padding=True, truncation=True, max_length=512)
    # Move input tensors to the same device as the model
    inputs = {k: v.to(model.device) for k, v in inputs.items()}
    with torch.no_grad():
        outputs = model(**inputs).logits
    preds = torch.argmax(outputs, dim = 1)
    return preds

texts = first_five_test_data['review']

baseline_preds = predict_sentiment(texts, model, tokenizer)

print("Baseline Predictions:", baseline_preds)
actual_prediction = first_five_test_data['sentiment'][0:5]
print("Actual Predictions:", actual_prediction)
```

Output

```
Baseline Predictions: tensor([1, 1, 0, 1, 1])
Actual Predictions: [1, 1, 0, 1, 0]
```

As we can see the base model predictions differ a little from the actual prediction.

Step 5: Apply PEFT Using LoRA

Configure and attach LoRA adapters to the pretrained model.

5.1 Configure LoRA Parameters

- Rank (r) – higher the rank the more complex pattern it can capture
- Alpha
- Target modules

```
lora_config = LoraConfig(  
    task_type = "SEQ_CLS",  
    r = 4,  
    #r = 8,  
    lora_alpha = 16,  
    #lora_alpha = 32,  
    target_modules = ["query", "value"],  
    lora_dropout = 0.1  
)
```

5.2 Attach LoRA Adapters

- Attach LoRA adapters to the pretrained model using the peft library

```
lora_model = get_peft_model(model, lora_config)
```

5.3 Verify Trainable Parameters

- Verify that only LoRA parameters are trainable
- Verify that base model parameters remain frozen

```
lora_model.print_trainable_parameters() # View the small percentage of updated weights
```

trainable params: 4,354 || all params: 4,390,532 || trainable%: 0.0992

Step 6: Fine-Tune the Model

Tokenize the training data, train the model, monitor loss, and save the fine-tuned model.

6.1 Tokenize Training Data

- Tokenize the training dataset for input to the model

```
def tokenize_data(sample_data):
    return tokenizer(sample_data["review"], padding=True, truncation=True, max_length=512)
```

```
train_dataset = imdb_split_dataset['train']
test_dataset = imdb_split_dataset['test']

tokenized_train_dataset = train_dataset.map(tokenize_data, batched=True)
tokenized_test_dataset = test_dataset.map(tokenize_data, batched=True)

tokenized_train_dataset = tokenized_train_dataset.rename_column("sentiment", "labels")
tokenized_test_dataset = tokenized_test_dataset.rename_column("sentiment", "labels")

tokenized_train_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])
tokenized_test_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])
```

Map: 100%  45000/45000 [00:23<00:00, 1992.75 examples/s]

Map: 100%  5000/5000 [00:02<00:00, 1935.84 examples/s]

6.2 Train the Model

- Train the model for a few epochs
- Monitor training loss

```
training_args = TrainingArguments(
    output_dir="./results",
    learning_rate = 2e-4,
    per_device_train_batch_size = 4,
    num_train_epochs = 1,
    weight_decay = 0.1,
    logging_steps = 10
    #remove_unused_columns = False # Changed to True to resolve padding conflict
)
```

```
trainer = Trainer(
    model=lora_model,
    args=training_args,
    train_dataset=tokenized_train_dataset,
    processing_class=tokenizer
)
```

```
trainer.train()
```

Monitoring Loss :

Step	Training Loss		
		5470	0.806712
10	1.387612	5480	0.799843
20	1.384344	5490	1.103441
30	1.397207	5500	0.950429
40	1.378388	5510	0.922543
50	1.376589	5520	0.920381
60	1.361819	5530	1.131361
70	1.381025	5540	0.873110
80	1.419858	5550	1.171416
90	1.361232	5560	0.784014
100	1.351358	5570	0.870912
110	1.338951	5580	0.816688
120	1.391337	5590	0.895607
130	1.368007	5600	0.857092
140	1.369845	5610	0.918061
150	1.378689		

6.3 Save the Fine-Tuned Model

- Save the fine-tuned LoRA model to disk

```
model.save_pretrained("./lora-sentiment-model")
print("Model trained and saved successfully.")
```

Writing model shards: 100%  1/1 [00:00<00:00, 17.87it/s]
Model trained and saved successfully.

```
tokenizer.save_pretrained("./lora-sentiment-model")
```

```
('./lora-sentiment-model/tokenizer_config.json',  
 './lora-sentiment-model/tokenizer.json')
```


Step 7: Model Evaluation

Evaluate the fine-tuned model on the test dataset and compare results with the baseline.

7.1 Evaluate on Test Dataset

- Run the fine-tuned model on the test dataset

```
finetuned_preds = predict_sentiment(texts, model, tokenizer)
print(finetuned_preds)
```

```
tensor([1, 1, 0, 1, 0], device='cuda:0')
```

7.2 Compare Before vs After Fine-Tuning

- Predictions before fine-tuning (from Step 4)
- Predictions after fine-tuning

```
sentiment_ds=pd.DataFrame({
    'Actual Prediction': actual_prediction,
    'Baseline Predictions': baseline_preds.cpu().tolist(),
    'Finetuned Predictions': finetuned_preds.cpu().tolist()
})
print(sentiment_ds)
```

	Actual Prediction	Baseline Predictions	Finetuned Predictions
0	1	1	1
1	1	1	1
2	0	0	0
3	1	1	1
4	0	1	0

Step 8: RLHF – Conceptual Explanation (No Coding)

8.1 What is RLHF?

RLHF is a LLM training technique which uses human feedback to train the model. Apart from using supervised fine tuning in the initial stage to optimize our pretrained model to generate responses, we use RLHF to refine our model responses based on user feedback. Good response by model- give it a thumbs up, bad response – thumbs down. Model will be learning from these ratings.

8.2 Why is RLHF Important for LLM Alignment?

Let's look at an unaligned pre-trained model – the LLM model is trained on huge amounts of data available on the internet basically to predict the next token. So, it is nothing but highly sophisticated prediction machine. But as user we don't just want to predict next words but want to make sense of our prompt/question to provide a response/answer which follows HHH (helpful, harmless, honest).

This is where the LLM alignment is necessary, we can use supervised fine tuning but it needs human intervention as we need to create a dataset that consists of pairs of prompts and responses. But, it comes with its own limitations, SFT can have only limited dataset, also it might not cover all the prompts that might be asked and model fails for unseen data.

This is where RLHF comes in; why not ask the humans whether they like or dislike the response from LLM and capture their feedback and train our model on a reward-based system so that we can have more real and acceptable answers to our prompts.

8.3 Comparison: Full Fine-Tuning vs PEFT (LoRA) vs RLHF

Full Fine-Tuning – Train all the parameters (weights and biases), requires huge computational resources.

PEFT (LoRA) – Append small set of trainable weights to pre-trained weights; requires less compute

RLHF – LLM model alignment technique using human feedback to train model.

Step 9: Conclusion

Summarize the project and reflect on what you have learned.

9.1 Project Summary

- Summarize the overall project and its objectives

Project Summarization-

- Using PEFT-LoRA , train a base model for sentiment analysis.
- Rather than training the entire model parameters, a small set of parameters are added, freezing the base model parameters.
- LoRA uses hyperparameters such as rank – higher the rank ,the more complex patterns will be learned.
- In short, if we face behavioural problem with our model, we can fine tune it using LoRA

Objective –

- Learn fine tuning using PEFT-LoRA.
- Understanding difference between full fine tuning vs PEFT vs RLHF

9.2 Benefits of PEFT & LoRA

As compared to full fine tuning, PEFT-LoRA uses

- less memory,
- lowers computational cost,
- prevents overfitting.

9.3 Real-World Applications

- Chatbots
- Review analysis
- Customer feedback systems
- Customer Support
- Domain specific Q&A